

A Report on

Student Academic Management System

Submitted in partial fulfilment for the
degree of Bachelor of Technology in
Artificial Intelligence

Submitted by
Mansi Rangne -58
Sakshi Wavhal - 74
Sharvari Khare - 76

Under the guidance of
Prof. Kumud Wasnik

Department of
Artificial Intelligence



Usha Mittal Institute of Technology (SNDT Women's University)

Juhu Tara Road
Santacruz West, Mumbai
2025-2026

Certificate

This is to certify that Sharvari Khare , Sakshi Wavhal and Mansi Rangne have completed the project report on the topic “Student Academic Management System” satisfactorily in partial fulfillment for the Bachelor’s Degree in Artificial Intelligence under the guidance of Prof.Kumud Wasnik during the year 2025-2026 as prescribed by Usha Mittal Institute of Technology (SNDT Women’s University).

Guide

Prof.Kumud Wasnik

Head of Department

Dr. Anita Morey

Principal

Dr. Yogesh P Nerkar

Declaration

I declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, I have adequately cited and referenced the original sources. I also declare that I have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in my submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

Abstract

The Student Academic Management System project is designed to efficiently organize, manage, and manipulate academic data using a structured relational database. The system consists of multiple interconnected tables, namely Student, Teacher, Department, Courses, Enrollment, and Grade, which together represent a comprehensive academic environment. Each table stores specific information such as student details, teacher profiles, course information, departmental data, enrollment records, and student performance, while maintaining relationships through keys to ensure data integrity and consistency.

The primary objective of this project is to develop a database system that supports fundamental operations such as insertion, updating, and deletion of records, enabling dynamic management of academic data. Users can add new students, assign teachers to courses, enroll students in different subjects, update records when changes occur, and remove outdated or incorrect data. These operations ensure that the database remains accurate, up-to-date, and reliable for institutional use.

In addition to basic operations, the system is designed to perform a wide range of SQL queries to retrieve meaningful information from the database. These include querying student enrollments, calculating grades, listing courses offered by departments, identifying teacher-course assignments, and generating reports based on academic performance. Advanced queries such as joins, aggregations, and filtering conditions are used to extract insights and support decision-making processes.

The project also emphasizes proper database design principles, including normalization, the use of primary and foreign keys, and maintaining relationships between tables to minimize redundancy and avoid anomalies. By structuring the data efficiently, the system ensures scalability, consistency, and ease of maintenance.

Overall, this project demonstrates the practical implementation of database management concepts and SQL operations, providing a robust solution for handling academic data in an organized and efficient manner.

Keywords: Student Academic Management System, Relational Database, SQL, Tables, Primary Key, Foreign Key, Normalization, Data Integrity, Enrollment, Grades, Database Design, CRUD Operations, Joins, Aggregation, Data Management

Contents

Abstract	4
1 Introduction	6
2 Problem Statement	7
3 Objectives	8
4 Relational Database Design	9
5 ER Diagram	10
6 Schema Diagram	11
7 Postgre SQL queries	12
8 Conclusion	24

1 Introduction

The Student Academic Management System is developed to efficiently manage and organize academic data using a relational database approach. This system is designed using six main tables, namely *Student*, *Teacher*, *Department*, *Courses*, *Enrollment*, and *Grade*, which together represent the core structure of an educational institution.

Each table contains important attributes such as *course_name*, *credits*, *dept_id*, *dept_name*, *semester*, *student_id*, *enrollment_id*, *marks*, *grade*, *grade_id*, *gender*, *city*, and *date_of_birth*, ensuring that all relevant information is stored in a structured and meaningful way. In this project, we have extensively used data definition language (ddl) and data manipulation language (dml) commands to build and manage the database. Ddl commands such as create, alter, rename, and drop are used to define and modify the database structure. A total of six tables have been created using the create command, and relationships between them are maintained using appropriate keys. Dml commands such as insert, update, delete, and select are used to manipulate and retrieve data from the database. We have inserted multiple records into each table to simulate real-world academic data.

Additionally, various compound conditions have been applied in sql queries to filter and retrieve specific data. These include operators such as and, or, like, not like, between, in, and not in, which help in performing complex queries efficiently. The project also makes use of group by and order by clauses to organize and present data in a meaningful format.

To perform data analysis, aggregate functions such as sum, avg, count, min, and max have been used to calculate total marks, average performance, and the number of records. Furthermore, advanced database features such as views and triggers have been implemented. Views are used to simplify complex queries and present data in a user-friendly manner, while triggers help in automatically executing specific actions when certain conditions are met, ensuring data integrity and consistency.

Overall, this project demonstrates the practical application of sql commands and database management concepts to efficiently handle academic data. It provides a strong foundation for understanding how real-world database systems are designed, managed, and optimized.

2 Problem Statement

Managing academic data manually or using unstructured systems often leads to issues such as data redundancy, inconsistency, and difficulty in retrieving meaningful information. Educational institutions handle large volumes of data related to students, teachers, courses, departments, enrollments, and grades, which require efficient organization and management. Without a proper database system, performing operations such as inserting new records, updating existing information, deleting outdated data, and generating reports becomes time-consuming and error-prone. Additionally, executing complex queries to analyze student performance or course details is difficult without a structured approach. Therefore, there is a need for a well-designed student database management system that ensures data accuracy, integrity, and efficient retrieval using SQL operations.

3 Objectives

The main objective of this project is to design and implement an efficient student academic management system that can handle academic data accurately and perform various sql operations effectively. It aims to provide a structured approach for managing and analyzing data using database concepts.

- To design and develop a student database management system using relational database concepts.
- To create and manage six tables: student, teacher, department, courses, enrollment, and grade.
- To implement data definition language (ddl) commands such as create, alter, rename, and drop for defining the database structure.
- To use data manipulation language (dml) commands such as insert, update, delete, and select for managing and retrieving data.
- To perform complex queries using conditions like and, or, like, not like, between, in, and not in.
- To organize and present data using group by and order by clauses.
- To apply aggregate functions such as sum, avg, count, min, and max for data analysis.
- To implement views for simplifying complex queries and improving data accessibility.
- To use triggers to maintain data integrity and automate specific database operations.
- To ensure efficient, accurate, and consistent management of academic data.

4 Relational Database Design

Database tables

1. Courses Table

- Attributes - course_id (Primary key), course_name , credits
- Purpose - This table stores detailed information about courses offered by the institution. The primary key ensures each course is uniquely identified, while the credits attribute helps in academic evaluation and workload calculation.

2. Departments table

- Attributes - dept_id , dept_name
- Purpose - This table maintains structured information about different departments. It helps in organizing courses and teachers under specific departments, ensuring proper data categorization and integrity.

3. Enrollments table

- Attributes - enrollment_id (Primary key) , student_id , course_id (Foreign key) , semester
- Purpose - This table represents the relationship between students and courses. It records which student is enrolled in which course and in which semester, thereby resolving the many-to-many relationship between students and courses.

4. Grades table

- Attributes - grades_id (Primary Key) , enrollment_id (Foreign Key), marks , grade
- Purpose - This table stores the performance details of students for each enrollment. It ensures that marks and grades are linked to specific enrollments, maintaining data consistency and enabling academic analysis.

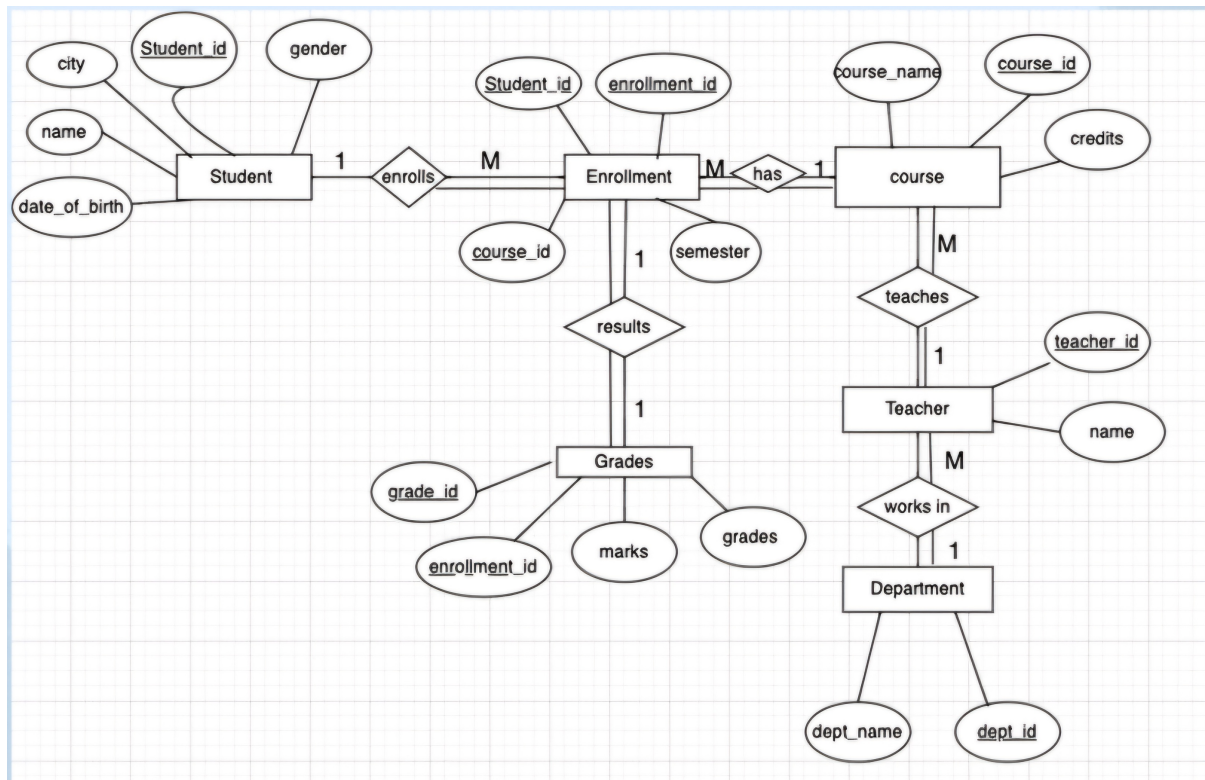
5. Student table

- Attributes - student_id (Primary key) , name , gender , city , date_of_birth
- Purpose - This table contains personal and demographic information of students. The primary key uniquely identifies each student, and the stored attributes support record management and reporting.

6. Teacher table

- Attributes - teacher_id (Primary key), name , department
- Purpose - This table stores details of teachers and their associated departments. It helps in managing faculty information and linking teachers to courses and departments within the system.

5 ER Diagram



6 Schema Diagram

- Courses table

course_id	course_name	credits

- Teacher table

teacher_id	name	department

- Enrollment table

enrollment_id	student_id	course_id	semester

- Grades table

grade_id	enrollment_id	marks	grade

- Student table

student_id	name	gender	city	date_of_birth

- Departments table

dept_id	dept_name

7 Postgre SQL queries

```
student_academic_db=# select * from courses;
```

course_id	course_name	credits
1	DBMS	4
2	Data Structures	3
3	Operating Systems	4
4	Computer Networks	3
5	Software Engineering	3
6	Machine Learning	4
7	Artificial Intelligence	4
8	Web Development	3
9	Cyber Security	3
10	Cloud Computing	4
11	Big Data	4
12	Data Mining	3
13	Compiler Design	4
14	Digital Logic	3
15	Microprocessors	4
16	Distributed Systems	4
17	Mobile Computing	3
18	Blockchain	3
19	IoT	3
20	Game Development	3
21	Python Programming	3
22	Java Programming	3
23	C Programming	3
24	Software Testing	3
25	UI UX Design	2
26	Computer Graphics	3
27	Numerical Methods	3
28	Linear Algebra	4
29	Statistics	3
30	Research Methodology	2
31	Ethical Hacking	3
32	Data Analytics	4
33	Deep Learning	4
34	Natural Language Processing	4
35	Quantum Computing	4

(35 rows)

(a) Courses Table

```
student_academic_db=# select * from teacher;
```

teacher_id	name	department
1	Dr. Rao	CS
2	Prof. Mehta	IT
3	Dr. Khan	CS
4	Prof. Iyer	ECE
5	Dr. Sharma	IT
6	Prof. Gupta	CS
7	Dr. Reddy	IT
8	Prof. Singh	ECE
9	Dr. Patel	CS
10	Prof. Nair	IT
11	Dr. Das	CS
12	Prof. Verma	IT
13	Dr. Mishra	ECE
14	Prof. Joshi	CS
15	Dr. Kulkarni	IT
16	Prof. Shah	ECE
17	Dr. Jain	CS
18	Prof. Yadav	IT
19	Dr. Roy	ECE
20	Prof. Kapoor	CS
21	Dr. Bansal	IT
22	Prof. Arora	ECE
23	Dr. Singh	CS
24	Prof. Malhotra	IT
25	Dr. Sinha	ECE
26	Prof. Chatterjee	CS
27	Dr. Nair	IT
28	Prof. Pillai	ECE
29	Dr. Saxena	CS
30	Prof. Tiwari	IT
31	Dr. Ghosh	ECE
32	Prof. Desai	CS
33	Dr. Bhatt	IT
34	Prof. Chauhan	ECE
35	Dr. Dubey	CS

(35 rows)

(b) Teacher Table

```
student_academic_db=# select * from enrollments;
```

enrollment_id	student_id	course_id	semester
1	101	1	Sem 1
2	102	2	Sem 1
3	103	3	Sem 1
4	104	4	Sem 1
5	105	5	Sem 1
6	106	6	Sem 1
7	107	7	Sem 1
8	108	8	Sem 1
9	109	9	Sem 1
10	110	10	Sem 1
11	111	11	Sem 1
12	112	12	Sem 1
13	113	13	Sem 1
14	114	14	Sem 1
15	115	15	Sem 1
16	116	16	Sem 2
17	117	17	Sem 2
18	118	18	Sem 2
19	119	19	Sem 2
20	120	20	Sem 2
21	121	21	Sem 2
22	122	22	Sem 2
23	123	23	Sem 2
24	124	24	Sem 2
25	125	25	Sem 2
26	126	26	Sem 2
27	127	27	Sem 2
28	128	28	Sem 2
29	129	29	Sem 2
30	130	30	Sem 2
31	131	31	Sem 3
32	132	32	Sem 3
33	133	33	Sem 3
34	134	34	Sem 3
35	135	35	Sem 3

(35 rows)

(c) Enrollments Table

```
student_academic_db=# select * from grades;
```

grade_id	enrollment_id	marks	grade
1	1	85	A
2	2	78	B
3	3	90	A
4	4	67	C
5	5	88	A
6	6	72	B
7	7	91	A
8	8	69	C
9	9	75	B
10	10	82	A
11	11	80	A
12	12	76	B
13	13	89	A
14	14	64	C
15	15	70	B
16	16	87	A
17	17	73	B
18	18	68	C
19	19	92	A
20	20	79	B
21	21	84	A
22	22	77	B
23	23	66	C
24	24	81	A
25	25	74	B
26	26	90	A
27	27	71	B
28	28	65	C
29	29	88	A
30	30	83	A
31	31	78	B
32	32	69	C
33	33	91	A
34	34	72	B
35	35	86	A

(35 rows)

(d) Grades Table

```
student_academic_db=# select * from student;
```

student_id	name	gender	city	date_of_birth
101	Rishi Patel	Female	Pune	2002-08-25
102	Rishi Verma	Male	Delhi	2003-01-18
103	Sanya Iyer	Female	Chennai	2002-11-02
104	Aryan Singh	Male	Chennai	2003-05-10
105	Prisha Mehta	Female	Amnabad	2002-03-28
106	Aanya Gupta	Female	Bombay	2003-07-29
107	Arjun Shah	Male	Bombay	2002-09-22
108	Arushi Ghosh	Female	Delhi	2002-12-05
109	Rishi Kumar	Male	Pune	2003-06-05
110	Neelika Nair	Female	Chennai	2002-05-19
111	Usha Yadav	Female	Lucknow	2003-02-15
112	Lijana Nair	Female	Chennai	2002-10-09
113	Aditya Joshi	Male	Kolkata	2003-03-24
114	Rohan Shah	Male	Bombay	2002-07-22
115	Samir Jha	Male	Delhi	2003-08-18
116	Rishi Sanyal	Male	Hyderabad	2002-09-18
117	Chaiti Ghosh	Female	Bombay	2003-01-28
118	Nikhila Roy	Female	Kolkata	2002-08-05
119	Neelika Nair	Female	Kolkata	2003-04-12
120	Arushi Singh	Female	Delhi	2002-11-27
121	Rishi Singh	Male	Delhi	2003-10-04
122	Arya Agarwal	Female	Delhi	2002-10-28
123	Harsh Varshma	Male	Pune	2003-12-05
124	Harsh Verma	Male	Amnabad	2002-02-28
125	Vaish Patel	Female	Lucknow	2003-01-09
126	Rishi Kulkarni	Male	Pune	2002-08-21
127	Neelika Nair	Female	Chandigarh	2003-06-19
128	Rishi Sen	Male	Kolkata	2002-08-19
129	Arushi Nair	Female	Kolkata	2002-07-04
130	Shreya Nair	Female	Kolkata	2002-08-14
131	Rohan Kumar	Male	Kolkata	2003-02-29
132	Lila Kumar	Female	Delhi	2002-10-18
133	Rishi Gupta	Male	Kolkata	2002-11-21
134	Aditya Jha	Male	Kolkata	2003-01-10
135	Raj Malhotra	Male	Bombay	2002-02-08
136	Rishi Sharma	Male	Pune	2003-05-14

(36 rows)

(e) Student Table

```
student_academic_db=# select * from departments;
```

dept_id	dept_name
1	Computer Science
2	Information Technology
3	Electronics
4	Mechanical
5	Civil
6	Electrical
7	Artificial Intelligence
8	Data Science
9	Cyber Security
10	Cloud Computing
11	Software Engineering
12	Robotics
13	Automobile Engineering
14	Aerospace Engineering
15	Biotechnology
16	Chemical Engineering
17	Environmental Engineering
18	Marine Engineering
19	Industrial Engineering
20	Textile Engineering
21	Agricultural Engineering
22	Mining Engineering
23	Petroleum Engineering
24	Nanotechnology
25	Physics
26	Mathematics
27	Statistics
28	Commerce
29	Management
30	Economics
31	Accounting
32	Finance
33	Marketing
34	Human Resources
35	Law

(35 rows)

(f) Departments Table

Figure 1: Database Tables

```
postgres=# CREATE database student_academicc_db;
CREATE DATABASE
postgres=# \c student_academicc_db;
You are now connected to database "student_academicc_db" as user "postgres".
student_academicc_db=#
```

Figure 2: Create a database

student_id	name	gender	city	date_of_birth
101	Amit Sharma	Male	Mumbai	2003-05-14
102	Neha Patel	Female	Pune	2002-08-21
103	Rahul Verma	Male	Delhi	2003-01-10
104	Sneha Iyer	Female	Chennai	2002-11-02
105	Arjun Singh	Male	Jaipur	2003-07-19
106	Priya Mehta	Female	Ahmedabad	2002-03-30
107	Karan Gupta	Male	Mumbai	2003-09-12
108	Anjali Desai	Female	Surat	2002-12-25
109	Rohit Kumar	Male	Patna	2003-06-05
110	Pooja Nair	Female	Kochi	2002-04-17
111	Vikas Yadav	Male	Lucknow	2003-02-11
112	Simran Kaur	Female	Amritsar	2002-10-09
113	Aditya Joshi	Male	Nagpur	2003-03-14
114	Meera Shah	Female	Mumbai	2002-07-22
115	Rakesh Jain	Male	Indore	2003-08-30
116	Divya Reddy	Female	Hyderabad	2002-09-18
117	Sahil Khan	Male	Bhopal	2003-04-28
118	Nikita Roy	Female	Kolkata	2002-05-05
119	Manish Das	Male	Guwahati	2003-06-12
120	Ayesha Shaikh	Female	Mumbai	2002-01-27
121	Deepak Singh	Male	Delhi	2003-11-03
122	Ritu Agarwal	Female	Jaipur	2002-06-15
123	Harsh Vardhan	Male	Patna	2003-12-01
124	Komal Verma	Female	Lucknow	2002-02-20
125	Yash Patel	Male	Ahmedabad	2003-03-09
126	Tanvi Kulkarni	Female	Pune	2002-04-11
127	Mohit Bansal	Male	Chandigarh	2003-05-19
128	Riya Sen	Female	Kolkata	2002-06-23
129	Abhishek Mishra	Male	Varanasi	2003-07-08
130	Shreya Nair	Female	Kochi	2002-08-14
131	Naveen Kumar	Male	Bangalore	2003-09-29
132	Isha Kapoor	Female	Delhi	2002-10-10
133	Aman Gupta	Male	Kanpur	2003-11-21
134	Pallavi Jain	Female	Indore	2002-12-12
135	Raj Malhotra	Male	Mumbai	2003-01-30
201	Aryan Singh	Male	Pune	2004-01-01

Figure 3: Select from Student table - Select operation is used to retrieve data from a database table

```
student_academicc_db=# INSERT INTO student (student_id, name, gender, city, date_of_birth) VALUES (101,'Amit Sharma','Male','Mumbai','2003-05-14'),(102,'Neha Patel','Female','Pune','2002-08-21'),(103,'Rahul Verma','Male','Delhi','2003-01-10'),(104,'Sneha Iyer','Female','Chennai','2002-11-02'),(105,'Arjun Singh','Male','Jaipur','2003-07-19'),(106,'Priya Mehta','Female','Ahmedabad','2002-03-30'),(107,'Karan Gupta','Male','Mumbai','2003-09-12'),(108,'Anjali Desai','Female','Surat','2002-12-25'),(109,'Rohit Kumar','Male','Patna','2003-06-05'),(110,'Pooja Nair','Female','Kochi','2002-04-17'),(111,'Vikas Yadav','Male','Lucknow','2003-02-11'),(112,'Simran Kaur','Female','Amritsar','2002-10-09'),(113,'Aditya Joshi','Male','Nagpur','2003-03-14'),(114,'Meera Shah','Female','Mumbai','2002-07-22'),(115,'Rakesh Jain','Male','Indore','2003-08-30'),(116,'Divya Reddy','Female','Hyderabad','2002-09-18'),(117,'Sahil Khan','Male','Bhopal','2003-04-28'),(118,'Nikita Roy','Female','Kolkata','2002-05-05'),(119,'Manish Das','Male','Guwahati','2003-06-12'),(120,'Ayesha Shaikh','Female','Mumbai','2002-01-27'),(121,'Deepak Singh','Male','Delhi','2003-11-03'),(122,'Ritu Agarwal','Female','Jaipur','2002-06-15'),(123,'Harsh Vardhan','Male','Patna','2003-12-01'),(124,'Komal Verma','Female','Lucknow','2002-02-20'),(125,'Yash Patel','Male','Ahmedabad','2003-03-09'),(126,'Tanvi Kulkarni','Female','Pune','2002-04-11'),(127,'Mohit Bansal','Male','Chandigarh','2003-05-19'),(128,'Riya Sen','Female','Kolkata','2002-06-23'),(129,'Abhishek Mishra','Male','Varanasi','2003-07-08'),(130,'Shreya Nair','Female','Kochi','2002-08-14'),(131,'Naveen Kumar','Male','Bangalore','2003-09-29'),(132,'Isha Kapoor','Female','Delhi','2002-10-10'),(133,'Aman Gupta','Male','Kanpur','2003-11-21'),(134,'Pallavi Jain','Female','Indore','2002-12-12'),(135,'Raj Malhotra','Male','Mumbai','2003-01-30');
INSERT 0 35
```

Figure 4: Insert values into Student table

```
student_academicc_db=# INSERT INTO grades (grade_id, enrollment_id, marks, grade) VALUES (1,1,85,'A'),(2,2,78,'B'),(3,3,90,'A'),(4,4,67,'C'),(5,5,88,'A'),(6,6,72,'B'),(7,7,91,'A'),(8,8,69,'C'),(9,9,75,'B'),(10,10,82,'A'),(11,11,80,'A'),(12,12,76,'B'),(13,13,89,'A'),(14,14,64,'C'),(15,15,70,'B'),(16,16,87,'A'),(17,17,73,'B'),(18,18,68,'C'),(19,19,92,'A'),(20,20,79,'B'),(21,21,84,'A'),(22,22,77,'B'),(23,23,66,'C'),(24,24,81,'A'),(25,25,74,'B'),(26,26,90,'A'),(27,27,71,'B'),(28,28,65,'C'),(29,29,88,'A'),(30,30,83,'A'),(31,31,78,'B'),(32,32,69,'C'),(33,33,91,'A'),(34,34,72,'B'),(35,35,86,'A');
INSERT 0 35
```

Figure 5: Insert into Grades table

```
student_academicc_db=# INSERT INTO courses (course_id, course_name, credits) VALUES (1,'DBMS',4),(2,'Data Structures',3),(3,'Operating Systems',4),(4,'Computer Networks',3),(5,'Software Engineering',3),(6,'Machine Learning',4),(7,'Artificial Intelligence',4),(8,'Web Development',3),(9,'Cyber Security',3),(10,'Cloud Computing',4),(11,'Big Data',4),(12,'Data Mining',3),(13,'Compiler Design',4),(14,'Digital Logic',3),(15,'Microprocessors',4),(16,'Distributed Systems',4),(17,'Mobile Computing',3),(18,'Blockchain',3),(19,'IoT',3),(20,'Game Development',3),(21,'Python Programming',3),(22,'Java Programming',3),(23,'C Programming',3),(24,'Software Testing',3),(25,'UI UX Design',2),(26,'Computer Graphics',3),(27,'Numerical Methods',3),(28,'Linear Algebra',4),(29,'Statistics',3),(30,'Research Methodology',2),(31,'Ethical Hacking',3),(32,'Data Analytics',4),(33,'Deep Learning',4),(34,'Natural Language Processing',4),(35,'Quantum Computing',4);
INSERT 0 35
```

Figure 6: Insert into courses table.

```
student_academic_db=# ALTER TABLE student DROP COLUMN phone;
ALTER TABLE
student_academic_db=# select * from student;
```

student_id	name	gender	city	date_of_birth
101	Amit Sharma	Male	Mumbai	2003-05-14
102	Neha Patel	Female	Pune	2002-08-21
103	Rahul Verma	Male	Delhi	2003-01-10
104	Sneha Iyer	Female	Chennai	2002-11-02
105	Arjun Singh	Male	Jaipur	2003-07-19
106	Priya Mehta	Female	Ahmedabad	2002-03-30
107	Karan Gupta	Male	Mumbai	2003-09-12
108	Anjali Desai	Female	Surat	2002-12-25
109	Rohit Kumar	Male	Patna	2003-06-05
110	Pooja Nair	Female	Kochi	2002-04-17

Figure 7: Drop command is used to remove an existing attribute from a table.

```
student_academic_db=# ALTER TABLE student ADD phone VARCHAR(15);
ALTER TABLE
student_academic_db=# select * from student;
```

student_id	name	gender	city	date_of_birth	phone
101	Amit Sharma	Male	Mumbai	2003-05-14	
102	Neha Patel	Female	Pune	2002-08-21	
103	Rahul Verma	Male	Delhi	2003-01-10	
104	Sneha Iyer	Female	Chennai	2002-11-02	
105	Arjun Singh	Male	Jaipur	2003-07-19	
106	Priya Mehta	Female	Ahmedabad	2002-03-30	
107	Karan Gupta	Male	Mumbai	2003-09-12	
108	Anjali Desai	Female	Surat	2002-12-25	

Figure 8: This command is used to add a new column to an existing table.

```
student_academic_db=# ALTER TABLE teachers RENAME TO teacher;
ALTER TABLE
student_academic_db=# select * from teacher;
```

teacher_id	name	department
1	Dr. Rao	CS
2	Prof. Mehta	IT
3	Dr. Khan	CS
4	Prof. Iyer	ECE
5	Dr. Sharma	IT
6	Prof. Gupta	CS
7	Dr. Reddy	IT
8	Prof. Singh	ECE
9	Dr. Patel	CS
10	Prof. Nair	IT
11	Dr. Das	CS
12	Prof. Verma	IT
13	Dr. Mishra	ECE
14	Prof. Joshi	CS

Figure 9: The Rename command is used to change the name of an existing database object

```
student_academic_db=# UPDATE student SET city='Nagpur' WHERE student_id=101;
UPDATE 1
```

Figure 10: Update command is used to modify existing records in a table.

```
student_academicc_db=# DELETE FROM student WHERE student_id=201;
DELETE 1
student_academicc_db=# select * from student;
```

student_id	name	gender	city	date_of_birth
102	Neha Patel	Female	Pune	2002-08-21
103	Rahul Verma	Male	Delhi	2003-01-10
104	Sneha Iyer	Female	Chennai	2002-11-02
105	Arjun Singh	Male	Jaipur	2003-07-19
106	Priya Mehta	Female	Ahmedabad	2002-03-30
107	Karan Gupta	Male	Mumbai	2003-09-12
108	Anjali Desai	Female	Surat	2002-12-25
109	Rohit Kumar	Male	Patna	2003-06-05
110	Pooja Nair	Female	Kochi	2002-04-17
111	Vikas Yadav	Male	Lucknow	2003-02-11
112	Simran Kaur	Female	Amritsar	2002-10-09
113	Aditya Joshi	Male	Nagpur	2003-03-14
114	Meera Shah	Female	Mumbai	2002-07-22
115	Rakesh Jain	Male	Indore	2003-08-30
116	Divya Reddy	Female	Hyderabad	2002-09-18
117	Sahil Khan	Male	Bhopal	2003-04-28
118	Nikita Roy	Female	Kolkata	2002-05-05
119	Manish Das	Male	Guwahati	2003-06-12
120	Ayesha Sheikh	Female	Mumbai	2002-01-27
121	Deepak Singh	Male	Delhi	2003-11-03
122	Ritu Agarwal	Female	Jaipur	2002-06-15
123	Harsh Vardhan	Male	Patna	2003-12-01
124	Komal Verma	Female	Lucknow	2002-02-20
125	Yash Patel	Male	Ahmedabad	2003-03-09
126	Tanvi Kulkarni	Female	Pune	2002-04-11
127	Mohit Bansal	Male	Chandigarh	2003-05-19
128	Riya Sen	Female	Kolkata	2002-06-23
129	Abhishek Mishra	Male	Varanasi	2003-07-08
130	Shreya Nair	Female	Kochi	2002-08-14
131	Naveen Kumar	Male	Bangalore	2003-09-29
132	Isha Kapoor	Female	Delhi	2002-10-10
133	Aman Gupta	Male	Kanpur	2003-11-21
134	Pallavi Jain	Female	Indore	2002-12-12
135	Raj Malhotra	Male	Mumbai	2003-01-30
101	Amit Sharma	Male	Nagpur	2003-05-14

(35 rows)

Figure 11: Delete command is used to remove specific records from a table.

```
student_academicc_db=# SELECT * FROM courses WHERE credits=4 AND course_name LIKE 'D%';
```

course_id	course_name	credits
1	DBMS	4
16	Distributed Systems	4
32	Data Analytics	4
33	Deep Learning	4

(4 rows)

Figure 12: AND is used to combine multiple conditions and returns results only when all the conditions are true.

```
student_academicc_db=# SELECT * FROM student WHERE city='Pune' OR city='Jaipur';
```

student_id	name	gender	city	date_of_birth
102	Neha Patel	Female	Pune	2002-08-21
105	Arjun Singh	Male	Jaipur	2003-07-19
122	Ritu Agarwal	Female	Jaipur	2002-06-15
126	Tanvi Kulkarni	Female	Pune	2002-04-11

(4 rows)

Figure 13: OR is used to combine conditions and returns results when at least one condition is true.

```
student_academic_db=# SELECT * FROM teacher WHERE department IN ('ECE');
 teacher_id |      name      | department
-----+-----+-----
          4 | Prof. Iyer     | ECE
          8 | Prof. Singh    | ECE
         13 | Dr. Mishra     | ECE
         16 | Prof. Shah     | ECE
         19 | Dr. Roy        | ECE
         22 | Prof. Arora    | ECE
         25 | Dr. Sinha      | ECE
         28 | Prof. Pillai   | ECE
         31 | Dr. Ghosh      | ECE
         34 | Prof. Chauhan  | ECE
(10 rows)
```

Figure 14: IN is used to check whether a value exists within a given list of values.

```
student_academic_db=# SELECT * FROM courses WHERE course_name NOT IN ('DBMS','Operating Systems','Machine Learning');
 course_id |      course_name      | credits
-----+-----+-----
          2 | Data Structures       | 3
          4 | Computer Networks     | 3
          5 | Software Engineering  | 3
          7 | Artificial Intelligence | 4
          8 | Web Development       | 3
          9 | Cyber Security        | 3
         10 | Cloud Computing       | 4
         11 | Big Data              | 4
         12 | Data Mining           | 3
         13 | Compiler Design       | 4
         14 | Digital Logic         | 3
         15 | Microprocessors       | 4
         16 | Distributed Systems   | 4
         17 | Mobile Computing      | 3
         18 | Blockchain            | 3
         19 | IoT                   | 3
         20 | Game Development      | 3
         21 | Python Programming    | 3
         22 | Java Programming      | 3
         23 | C Programming         | 3
         24 | Software Testing      | 3
         25 | UI UX Design          | 2
         26 | Computer Graphics     | 3
         27 | Numerical Methods     | 3
         28 | Linear Algebra        | 4
         29 | Statistics            | 3
         30 | Research Methodology  | 2
         31 | Ethical Hacking       | 3
         32 | Data Analytics        | 4
         33 | Deep Learning         | 4
         34 | Natural Language Processing | 4
         35 | Quantum Computing     | 4
(32 rows)
```

Figure 15: NOT IN is used to check whether a value does not exist within a given list of values.

```
student_academicc_db=# SELECT * FROM grades WHERE marks BETWEEN 85 AND 92;
grade_id | enrollment_id | marks | grade
-----+-----+-----+-----
      1 |              1 |     85 | A
      3 |              3 |     90 | A
      5 |              5 |     88 | A
      7 |              7 |     91 | A
     13 |             13 |     89 | A
     16 |             16 |     87 | A
     19 |             19 |     92 | A
     26 |             26 |     90 | A
     29 |             29 |     88 | A
     33 |             33 |     91 | A
     35 |             35 |     86 | A
(11 rows)
```

Figure 16: BETWEEN is used to select values that lie within a specified range, including the boundary values.

```
student_academicc_db=# SELECT * FROM student WHERE name LIKE 'A%';
student_id | name | gender | city | date_of_birth
-----+-----+-----+-----+-----
      105 | Arjun Singh | Male | Jaipur | 2003-07-19
      108 | Anjali Desai | Female | Surat | 2002-12-25
      113 | Aditya Joshi | Male | Nagpur | 2003-03-14
      120 | Ayesha Sheikh | Female | Mumbai | 2002-01-27
      129 | Abhishek Mishra | Male | Varanasi | 2003-07-08
      133 | Aman Gupta | Male | Kanpur | 2003-11-21
      101 | Amit Sharma | Male | Nagpur | 2003-05-14
(7 rows)
```

Figure 17: LIKE operator is used to search patterns in text data.

```
student_academicc_db=# GRANT SELECT, INSERT ON student TO PUBLIC;
GRANT
student_academicc_db=# REVOKE INSERT ON student FROM PUBLIC;
REVOKE
```

Figure 18: GRANT command is used to give specific permissions or privileges to users on database objects. REVOKE command is used to remove previously granted permissions or privileges from users.

```
student_academicc_db=# SELECT SUM(marks) AS total_marks FROM grades WHERE marks BETWEEN 70 AND 90;
total_marks
-----
      2008
(1 row)
```

Figure 19: SUM command is used to calculate the total sum of values in a numeric column.

```
student_academicc_db=# select city, COUNT(*) from student group by city having city = 'Mumbai' OR city = 'Pune' OR city = 'Delhi';
city | count
-----+-----
Delhi | 3
Mumbai | 4
Pune | 2
(3 rows)
```

Figure 20: COUNT command is used to count the number of rows or records in a table.

```
student_academicc_db=# SELECT grade, MIN(marks) AS lowest_marks FROM grades GROUP BY grade;
grade | lowest_marks
-----+-----
B      |             70
C      |             64
A      |             80
(3 rows)
```

Figure 21: MIN command is used to find the smallest value in a column.

```
student_academicc_db=# SELECT grade, MAX(marks) AS highest_marks FROM grades GROUP BY grade;
grade | highest_marks
-----+-----
B      |             79
C      |             69
A      |             92
(3 rows)
```

Figure 22: MAX command is used to find the largest value in a column.

```
student_academicc_db=# CREATE VIEW student_course_view AS SELECT s.name, c.course_name FROM student s JOIN enrollments e ON
s.student_id = e.student_id JOIN courses c ON e.course_id = c.course_id;
CREATE VIEW
student_academicc_db=# SELECT * FROM student_course_view LIMIT 5;
name | course_name
-----+-----
Amit Sharma | DBMS
Neha Patel | Data Structures
Rahul Verma | Operating Systems
Sneha Iyer | Computer Networks
Arjun Singh | Software Engineering
(5 rows)
```

Figure 23: JOIN operation is used to combine rows from two or more tables based on a related column between them.

```
student_academicc_db=# SELECT s.name, e.course_id FROM student s LEFT JOIN enrollments e ON s.student_id
= e.student_id WHERE s.student_id BETWEEN 101 AND 110 LIMIT 5;
name | course_id
-----+-----
Amit Sharma | 1
Neha Patel | 2
Rahul Verma | 3
Sneha Iyer | 4
Arjun Singh | 5
(5 rows)
```

Figure 24: LEFT JOIN is used to return all records from the left table and the matched records from the right table, with NULLs for non-matching rows.

```
student_academicc_db=# SELECT s.name, e.course_id FROM student s RIGHT JOIN enrollments e ON s.student_i
d = e.student_id WHERE e.course_id <= 5;
name | course_id
-----+-----
Amit Sharma | 1
Neha Patel | 2
Rahul Verma | 3
Sneha Iyer | 4
Arjun Singh | 5
(5 rows)
```

Figure 25: RIGHT JOIN is used to return all records from the right table and the matched records from the left table, with NULLs for non-matching rows.

```
student_academic_db=# SELECT s.name, e.course_id FROM student s FULL OUTER JOIN enrollments e ON s.student_id = e.student_id WHERE s.student_id BETWEEN 101 AND 110 OR e.course_id <= 5;
```

name	course_id
Amit Sharma	1
Neha Patel	2
Rahul Verma	3
Sneha Iyer	4
Arjun Singh	5
Priya Mehta	6
Karan Gupta	7
Anjali Desai	8
Rohit Kumar	9
Pooja Nair	10

(10 rows)

Figure 26: FULL OUTER JOIN is used to return all records when there is a match in either left or right table, including non-matching rows from both tables.

```
student_academic_db=# SELECT grade, COUNT(*) FROM grades GROUP BY grade HAVING COUNT(*) > 5;
```

grade	count
B	12
C	7
A	16

(3 rows)

Figure 27: GROUP BY clause is used to group rows that have the same values in specified columns into aggregated data sets. HAVING clause is used to filter grouped records based on aggregate conditions after the GROUP BY operation.

```
student_academic_db=# SELECT * FROM student ORDER BY student_id ASC;
```

student_id	name	gender	city	date_of_birth
101	Amit Sharma	Male	Nagpur	2003-05-14
102	Neha Patel	Female	Pune	2002-08-21
103	Rahul Verma	Male	Delhi	2003-01-10
104	Sneha Iyer	Female	Chennai	2002-11-02
105	Arjun Singh	Male	Jainpur	2002-07-18

Figure 28: ORDER BY clause is used to sort the result set based on one or more columns

```
student_academic_db=# SELECT name, city FROM student ORDER BY city ASC LIMIT 5;
```

name	city
Priya Mehta	Ahmedabad
Yash Patel	Ahmedabad
Simran Kaur	Amritsar
Naveen Kumar	Bangalore
Sahil Khan	Bhopal

(5 rows)

Figure 29: ORDER BY ASCENDING is used to sort the result set in increasing order (default).

```
student_academicc_db=# SELECT name, date_of_birth FROM student ORDER BY date_of_birth DESC LIMIT 5;
 name      | date_of_birth 
-----+-----
 Harsh Vardhan | 2003-12-01
 Aman Gupta   | 2003-11-21
 Deepak Singh | 2003-11-03
 Naveen Kumar | 2003-09-29
 Karan Gupta  | 2003-09-12
(5 rows)
```

Figure 30: ORDER BY DESCENDING is used to sort the result set in decreasing order.

```
student_academicc_db=# CREATE TABLE test_pk (id INT PRIMARY KEY, name VARCHAR(50));
CREATE TABLE
student_academicc_db=# \d test_pk
Table "public.test_pk"
Column | Type          | Collation | Nullable | Default
-----+-----+-----+-----+-----
 id     | integer       |           | not null |
 name   | character varying(50) |           |         |
Indexes:
    "test_pk_pkey" PRIMARY KEY, btree (id)
```

Figure 31: B-Tree Index is a balanced tree data structure used to improve the speed of data retrieval operations in a database.

```
student_academicc_db=# DROP INDEX idx_student_name;
DROP INDEX
student_academicc_db=# SELECT indexname FROM pg_indexes WHERE tablename='student'
indexname
-----
 student_pkey
 idx_student_city_gender
(2 rows)
```

Figure 32: DROP INDEX command is used to remove an existing index from a database table, thereby eliminating the associated data structure used for faster data retrieval.

```
student_academicc_db=# CREATE INDEX idx_student_city_gender ON student(city, gender);
CREATE INDEX
student_academicc_db=# CREATE INDEX idx_student_city_gender ON student(city, gender);
ERROR:  relation "idx_student_city_gender" already exists
student_academicc_db=# SELECT indexname FROM pg_indexes WHERE tablename='student' LIMIT 5;
indexname
-----
 student_pkey
 idx_student_name
 idx_student_city_gender
(3 rows)
```

Figure 33: CREATE INDEX command is used to create an index on one or more columns to enhance query performance.

```

student_academicc_db=# CREATE UNIQUE INDEX idx_course_name ON courses(course_name);
CREATE INDEX
student_academicc_db=# SELECT indexname FROM pg_indexes WHERE tablename='courses';
    indexname
-----
courses_pkey
idx_course_name
(2 rows)

```

Figure 34: UNIQUE INDEX is an index that enforces uniqueness of values in the indexed column

```

student_academicc_db=# CREATE VIEW student_view AS SELECT student_id, name, city FROM student;
CREATE VIEW
student_academicc_db=# SELECT * FROM student_view LIMIT 5;
 student_id |   name   | city
-----+-----+-----
          102 | Neha Patel | Pune
          103 | Rahul Verma | Delhi
          104 | Sneha Iyer | Chennai
          105 | Arjun Singh | Jaipur
          106 | Priya Mehta | Ahmedabad
(5 rows)

```

Figure 35: VIEW is a virtual table based on the result of a SQL query that does not store data physically.

```

student_academicc_db=# CREATE OR REPLACE FUNCTION log_update() RETURNS TRIGGER AS $$ BEGIN INSERT INTO student_log(message)
VALUES ('Updated student: ' || NEW.name); RETURN NEW; END; $$ LANGUAGE plpgsql;
CREATE FUNCTION
student_academicc_db=# CREATE TRIGGER student_update_trigger AFTER UPDATE ON student FOR EACH ROW EXECUTE FUNCTION log_update();
CREATE TRIGGER
student_academicc_db=# UPDATE student SET city='Pune' WHERE student_id=101;
UPDATE 1
student_academicc_db=# SELECT * FROM student_log;
 log_id | message
-----+-----
          1 | New student added: Test User
          2 | Updated student: Amit Sharma
(2 rows)

```

Figure 36: UPDATE TRIGGER is a database trigger that automatically executes a specified action after or before an UPDATE operation.

```

student_academicc_db=# CREATE TABLE student_log (log_id SERIAL PRIMARY KEY, message TEXT);
CREATE TABLE
student_academicc_db=# CREATE OR REPLACE FUNCTION log_student_insert() RETURNS TRIGGER AS $$ BEGIN INSERT INTO student_log(message) VALUES ('New student added: ' || NEW.name); RETURN NEW; END; $$ LANGUAGE plpgsql;
CREATE FUNCTION
student_academicc_db=# CREATE TRIGGER student_insert_trigger AFTER INSERT ON student FOR EACH ROW EXECUTE FUNCTION log_student_insert();
CREATE TRIGGER
student_academicc_db=#

```

Figure 37: INSERT TRIGGER is a database trigger that automatically executes a specified action after or before an INSERT operation.

```

student_academicc_db=# CREATE OR REPLACE FUNCTION log_delete() RETURNS TRIGGER AS $$ BEGIN INSERT INTO student_log(message)
VALUES ('Deleted student: ' || OLD.name); RETURN OLD; END; $$ LANGUAGE plpgsql;
CREATE FUNCTION
student_academicc_db=# CREATE TRIGGER student_delete_trigger AFTER DELETE ON student FOR EACH ROW EXECUTE FUNCTION log_delet
e();
ERROR: trigger "student_delete_trigger" for relation "student" already exists
student_academicc_db=# CREATE TRIGGER student_delete_trigger AFTER DELETE ON student FOR EACH ROW EXECUTE FUNCTION log_delet
e();
ERROR: trigger "student_delete_trigger" for relation "student" already exists
student_academicc_db=# SELECT * FROM student_log;
 log_id | message
-----+-----
      1 | New student added: Test User
      2 | Updated student: Amit Sharma
(2 rows)

```

Figure 38: DELETE TRIGGER is a database trigger that automatically executes a specified set of SQL statements before or after a DELETE operation on a table.

```

student_academicc_db=# CREATE TABLE test_check (marks INT CHECK (marks BETWEEN 0 AND 100));
CREATE TABLE
student_academicc_db=# INSERT INTO test_check VALUES (150);
ERROR: new row for relation "test_check" violates check constraint "test_check_marks_check"
DETAIL: Failing row contains (150).

```

Figure 39: CHECK constraint is used to enforce a condition on a column so that only values satisfying the specified condition are allowed in the table.

```

student_academicc_db=# EXPLAIN SELECT * FROM student WHERE name='Amit Sharma';
QUERY PLAN
-----
Seq Scan on student (cost=0.00..1.45 rows=1 width=282)
  Filter: ((name)::text = 'Amit Sharma'::text)
(2 rows)

```

Figure 40: EXPLAIN command is used to display the execution plan of a SQL query, showing how the database will access data and execute the query.

```

student_academicc_db=# SELECT s.name, c.course_name FROM student s INNER JOIN enrollments e ON s.student
_id = e.student_id INNER JOIN courses c ON e.course_id = c.course_id WHERE c.course_id <= 5;
 name | course_name
-----+-----
Amit Sharma | DBMS
Neha Patel | Data Structures
Rahul Verma | Operating Systems
Sneha Iyer | Computer Networks
Arjun Singh | Software Engineering
(5 rows)

student_academicc_db=# SELECT * FROM student LIMIT 5;
 student_id | name | gender | city | date_of_birth
-----+-----+-----+-----+-----
102 | Neha Patel | Female | Pune | 2002-08-21
103 | Rahul Verma | Male | Delhi | 2003-01-10
104 | Sneha Iyer | Female | Chennai | 2002-11-02
105 | Arjun Singh | Male | Jaipur | 2003-07-19
106 | Priya Mehta | Female | Ahmedabad | 2002-03-30
(5 rows)

```

Figure 41: INNER JOIN is used to retrieve only those records that have matching values in both tables based on a specified condition.

```

student_academic_db=# CREATE TABLE student_1nf (student_id INT, name VARCHAR(50), course VARCHAR(50));
CREATE TABLE
student_academic_db=# INSERT INTO student_1nf VALUES (1,'Amit','DBMS'),(1,'Amit','OS'),(2,'Neha','ML'),(3,'Rahul','CN'),(4,
'Sneha','AI');
INSERT 0 5
student_academic_db=# SELECT * FROM student_1nf LIMIT 5;
 student_id | name | course
-----+-----+-----
          1 | Amit | DBMS
          1 | Amit | OS
          2 | Neha | ML
          3 | Rahul | CN
          4 | Sneha | AI
(5 rows)

```

Figure 42: A table is in 1NF if it contains only atomic (indivisible) values and each field holds a single value with no repeating groups or multivalued attributes.

```

student_academic_db=# CREATE TABLE department_3nf (dept_id INT PRIMARY KEY, dept_name VARCHAR(50));
CREATE TABLE
student_academic_db=# CREATE TABLE student_3nf (student_id INT PRIMARY KEY, name VARCHAR(50), dept_id INT REFERENCES department_3nf(dept_id));
CREATE TABLE
student_academic_db=# INSERT INTO department_3nf VALUES (1,'CS'),(2,'IT'),(3,'ECE'),(4,'MECH'),(5,'CIVIL');
INSERT 0 5
student_academic_db=# INSERT INTO student_3nf VALUES (1,'Amit',1),(2,'Neha',2),(3,'Rahul',1),(4,'Sneha',3),(5,'Arjun',2);
INSERT 0 5
student_academic_db=# SELECT * FROM student_3nf;
 student_id | name | dept_id
-----+-----+-----
          1 | Amit |      1
          2 | Neha |      2
          3 | Rahul |      1
          4 | Sneha |      3
          5 | Arjun |      2
(5 rows)

student_academic_db=# SELECT * FROM department_3nf;
 dept_id | dept_name
-----+-----
        1 | CS
        2 | IT
        3 | ECE
        4 | MECH
        5 | CIVIL
(5 rows)

```

Figure 43: A table is in 3NF if it is in 2NF and has no transitive dependency, meaning non-key attributes depend only on the primary key and not on other non-key attributes.

8 Conclusion

The Student Database Management System successfully addresses the challenges of managing academic data by providing a structured and efficient solution. As discussed in the problem statement, traditional methods of handling data often lead to redundancy, inconsistency, and difficulty in data retrieval. This project overcomes these issues by using a well-designed relational database consisting of tables such as student, teacher, department, courses, enrollment, and grade.

The objectives of the project have been effectively achieved by implementing various sql operations, including ddl and dml commands, to create, manage, and manipulate the database. The use of conditions, aggregate functions, and clauses like group by and order by enables efficient data retrieval and analysis. Additionally, advanced features such as views and triggers enhance data accessibility, automation, and integrity.

Overall, this project demonstrates the practical application of database management concepts and provides a reliable system for storing, updating, and analyzing academic data. It serves as a strong foundation for understanding real-world database systems and highlights the importance of structured data management in educational institutions.